

Combined Logistic - Python

May 18, 2026

```
[1]: import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegressionCV
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import (confusion_matrix, classification_report,
                             roc_auc_score, roc_curve, accuracy_score,
                             precision_score, recall_score, f1_score)
from sklearn.feature_selection import VarianceThreshold
import matplotlib.pyplot as plt
import warnings
warnings.filterwarnings('ignore')
```

```
[2]: #read data
extr_data = pd.read_csv("extrinsic_dataset.csv")
intr_data = pd.read_csv("intrinsic_dataset.csv")

combined_data = pd.merge(intr_data, extr_data, on = ["Scientific", "Extinct"],
    ↪how="inner")

#for later:
risk = pd.read_csv("risk_scores.csv")

#basic info
print("Data shape:", combined_data.shape)
print("\nFirst few rows:")
print(combined_data.head())
print("\nData summary:")
print(combined_data.describe())
print(combined_data.columns.tolist())
print(combined_data.dtypes.tolist())
```

Data shape: (4890, 44)

First few rows:

	Scientific	Extinct	INTR_maximum_longevity_years	\
0	abrocoma boliviensis	0	6.372206	
1	abronia anzueto	0	21.041330	

2	abronia aurita	0	21.041330
3	abronia campbelli	0	21.041330
4	abronia chiszari	0	21.041330

	INTR_female_maturity_days	INTR_litter_or_clutch_size_n	\
0	239.019275	1.775565	
1	1722.192577	6.337562	
2	1722.192577	6.337562	
3	1722.192577	6.337562	
4	1722.192577	6.337562	

	INTR_litters_or_clutches_per_y	INTR_gestation_days	INTR_incubation_days	\
0	2.096639	94.873987	27.988462	
1	0.936963	217.287563	161.892004	
2	0.936963	217.287563	161.892004	
3	0.936963	217.287563	161.892004	
4	0.936963	217.287563	161.892004	

	INTR_female_body_mass_g	INTR_birth_or_hatching_svl_cm	...	\
0	474.442465	11.023394	...	
1	51.965648	4.392341	...	
2	51.965648	4.199820	...	
3	51.965648	4.238871	...	
4	51.965648	3.109324	...	

	habitat_Wetlands (inland)	conservation_Education & awareness	\
0	0.0	0.0	
1	0.0	0.0	
2	0.0	0.0	
3	0.0	0.0	
4	0.0	0.0	

	conservation_Land/water management	conservation_Land/water protection	\
0	0.0	0.0	
1	0.0	1.0	
2	0.0	0.0	
3	1.0	1.0	
4	0.0	0.0	

	conservation_Law & policy	\
0	0.0	
1	0.0	
2	0.0	
3	1.0	
4	0.0	

	conservation_Livelihood, economic & other incentives	\
0	0.0	

1	1.0
2	0.0
3	0.0
4	0.0

	conservation_Species management	threat1_Energy production & mining \
0	0.0	0.0
1	0.0	0.0
2	0.0	0.0
3	0.0	0.0
4	0.0	0.0

	threat1_Geological events \
0	0.0
1	0.0
2	0.0
3	0.0
4	0.0

	threat1_Invasive & other problematic species, genes & diseases
0	0.0
1	0.0
2	0.0
3	0.0
4	0.0

[5 rows x 44 columns]

Data summary:

	Extinct	INTR_maximum_longevity_years	INTR_female_maturity_days \
count	4890.000000	4890.000000	4890.000000
mean	0.017382	14.945140	770.936004
std	0.130705	14.732562	768.522966
min	0.000000	0.083333	33.239800
25%	0.000000	7.131812	366.122016
50%	0.000000	11.786722	545.202000
75%	0.000000	18.503691	888.551536
max	1.000000	211.000000	7305.000000

	INTR_litter_or_clutch_size_n	INTR_litters_or_clutches_per_y \
count	4890.000000	4890.000000
mean	3.430257	1.840389
std	5.861301	1.692712
min	0.721765	0.120000
25%	1.617918	1.001094
50%	2.400000	1.388996
75%	3.500000	1.858404
max	156.000000	23.000000

	INTR_gestation_days	INTR_incubation_days	INTR_female_body_mass_g \
count	4890.000000	4890.000000	4.890000e+03
mean	89.175856	43.375842	1.219856e+04
std	61.441027	43.116810	1.740283e+05
min	10.000000	5.544842	3.000000e-01
25%	67.302303	17.135762	1.593786e+01
50%	77.455654	27.000000	8.226158e+01
75%	91.182558	56.929834	5.316839e+02
max	730.480000	518.500000	7.302550e+06

	INTR_birth_or_hatching_svl_cm	INTR_no_sex_maturity_days ... \
count	4890.000000	4890.000000 ...
mean	8.807616	1010.648466 ...
std	27.362492	1363.099867 ...
min	0.000000	60.880000 ...
25%	2.528731	417.517533 ...
50%	3.836593	705.793525 ...
75%	6.696104	1130.298035 ...
max	759.999000	32523.944457 ...

	habitat_Wetlands (inland)	conservation_Education & awareness \
count	4890.000000	4890.000000
mean	0.073006	0.289980
std	0.260173	0.453799
min	0.000000	0.000000
25%	0.000000	0.000000
50%	0.000000	0.000000
75%	0.000000	1.000000
max	1.000000	1.000000

	conservation_Land/water management	conservation_Land/water protection \
count	4890.000000	4890.000000
mean	0.486299	0.482413
std	0.499863	0.499742
min	0.000000	0.000000
25%	0.000000	0.000000
50%	0.000000	0.000000
75%	1.000000	1.000000
max	1.000000	1.000000

	conservation_Law & policy \
count	4890.000000
mean	0.240286
std	0.427301
min	0.000000
25%	0.000000
50%	0.000000

75%	0.000000
max	1.000000

	conservation_Livelihood, economic & other incentives \
count	4890.000000
mean	0.068712
std	0.252989
min	0.000000
25%	0.000000
50%	0.000000
75%	0.000000
max	1.000000

	conservation_Species management	threat1_Energy production & mining \
count	4890.000000	4890.000000
mean	0.243763	0.006339
std	0.429395	0.079376
min	0.000000	0.000000
25%	0.000000	0.000000
50%	0.000000	0.000000
75%	0.000000	0.000000
max	1.000000	1.000000

	threat1_Geological events \
count	4890.000000
mean	0.000409
std	0.020222
min	0.000000
25%	0.000000
50%	0.000000
75%	0.000000
max	1.000000

	threat1_Invasive & other problematic species, genes & diseases
count	4890.000000
mean	0.031902
std	0.175757
min	0.000000
25%	0.000000
50%	0.000000
75%	0.000000
max	1.000000

[8 rows x 43 columns]

['Scientific', 'Extinct', 'INTR_maximum_longevity_years',
 'INTR_female_maturity_days', 'INTR_litter_or_clutch_size_n',
 'INTR_litters_or_clutches_per_y', 'INTR_gestation_days', 'INTR_incubation_days',
 'INTR_female_body_mass_g', 'INTR_birth_or_hatching_svl_cm',

```

'INTR_no_sex_maturity_days', 'INTR_gestation_incubation_days',
'INTR_litter_clutch_size', 'INTR_litters_clutches_per_year',
'INTR_interbirth_interval_years_1', 'INTR_birth_weight_g',
'INTR_metabolic_rate_w', 'total_locations', 'habitat_Artificial - Aquatic',
'habitat_Artificial - Terrestrial', 'habitat_Caves & Subterranean Habitats (non-
aquatic)', 'habitat_Desert', 'habitat_Forest', 'habitat_Grassland',
'habitat_Introduced Vegetation', 'habitat_Marine Coastal/Supratidal',
'habitat_Marine Intertidal', 'habitat_Marine Neritic', 'habitat_Marine Oceanic',
'habitat_Other', 'habitat_Rocky Areas (e.g., inland cliffs, mountain peaks)',
'habitat_Savanna', 'habitat_Shrubland', 'habitat_Unknown', 'habitat_Wetlands
(inland)', 'conservation_Education & awareness', 'conservation_Land/water
management', 'conservation_Land/water protection', 'conservation_Law & policy',
'conservation_Livelihood, economic & other incentives', 'conservation_Species
management', 'threat1_Energy production & mining', 'threat1_Geological events',
'threat1_Invasive & other problematic species, genes & diseases']
[dtypes('O'), dtype('int64'), dtype('float64'), dtype('float64'),
dtype('float64'), dtype('float64'), dtype('float64'), dtype('float64'),
dtype('float64'), dtype('float64'), dtype('float64'), dtype('float64'),
dtype('float64'), dtype('float64'), dtype('float64'), dtype('float64'),
dtype('float64'), dtype('int64'), dtype('float64'), dtype('float64'),
dtype('float64'), dtype('float64'), dtype('float64'), dtype('float64'),
dtype('float64'), dtype('float64'), dtype('float64'), dtype('float64'),
dtype('float64'), dtype('float64'), dtype('float64'), dtype('float64'),
dtype('float64'), dtype('float64'), dtype('float64'), dtype('float64'),
dtype('float64'), dtype('float64'), dtype('float64'), dtype('float64')]

```

```

[3]: #convert categorical variables to "category" type
cols_to_convert = combined_data.columns[combined_data.columns.
↳get_loc('habitat_Artificial - Aquatic'):
combined_data.columns.
↳get_loc('threat1_Invasive & other problematic species, genes & diseases')+1]

cols_to_skip = ['total_locations'] #skip these (not categorical)

cols_to_convert = [col for col in cols_to_convert if col not in cols_to_skip]

#convert these to ints
combined_data['total_locations'] = combined_data['total_locations'].
↳astype('int')

for col in cols_to_convert:
    if col in combined_data.columns:
        combined_data[col] = combined_data[col].astype('category')

print(combined_data.columns.tolist())
print(combined_data.dtypes.tolist())

```



```
ordered=False, categories_dtype=float64), CategoricalDtype(categories=[0.0,
1.0], ordered=False, categories_dtype=float64),
CategoricalDtype(categories=[0.0, 1.0], ordered=False,
categories_dtype=float64), CategoricalDtype(categories=[0.0, 1.0],
ordered=False, categories_dtype=float64), CategoricalDtype(categories=[0.0,
1.0], ordered=False, categories_dtype=float64),
CategoricalDtype(categories=[0.0, 1.0], ordered=False,
categories_dtype=float64), CategoricalDtype(categories=[0.0, 1.0],
ordered=False, categories_dtype=float64), CategoricalDtype(categories=[0.0,
1.0], ordered=False, categories_dtype=float64)]
```

```
[4]: #split into train and test sets with stratification
np.random.seed(123)
train, test = train_test_split(combined_data, test_size=0.3,
                              stratify=combined_data['Extinct'],
                              random_state=123)

#double check proportions
print("Proportion of extinct in full data:", combined_data['Extinct'].
      ↪value_counts(normalize=True))
print("\nProportion of extinct in train:", train['Extinct'].
      ↪value_counts(normalize=True))
print("\nProportion of extinct in test:", test['Extinct'].
      ↪value_counts(normalize=True))

#non-predictor columns
exclude_cols = ['Scientific', 'Extinct', 'population_trend.description']

#predictor columns
predictor_cols = [col for col in train.columns if col not in exclude_cols]

#prep data for variance filtering
X_train_temp = pd.get_dummies(train[predictor_cols], drop_first=False)
X_train_temp = X_train_temp.apply(pd.to_numeric, errors='coerce').fillna(0)

#near-zero variance filtering
selector = VarianceThreshold(threshold=0.001) # Low threshold for near-zero
      ↪variance
selector.fit(X_train_temp)

#get columns to keep
variance_mask = selector.get_support()
cols_to_keep = X_train_temp.columns[variance_mask].tolist()

#ID removed columns
removed_cols = X_train_temp.columns[~variance_mask].tolist()
print("Zero/near-zero variance predictors removed:")
```



```

for col in removed_cols:
    print(f" - {col}")

print(f"\nRemaining predictors after filtering: {len(cols_to_keep)}")
print(predictor_cols)

```

Proportion of extinct in full data: Extinct

0 0.982618

1 0.017382

Name: proportion, dtype: float64

Proportion of extinct in train: Extinct

0 0.982472

1 0.017528

Name: proportion, dtype: float64

Proportion of extinct in test: Extinct

0 0.982958

1 0.017042

Name: proportion, dtype: float64

Zero/near-zero variance predictors removed:

- habitat_Introduced Vegetation_0.0
- habitat_Introduced Vegetation_1.0
- habitat_Unknown_0.0
- habitat_Unknown_1.0
- threat1_Geological events_0.0
- threat1_Geological events_1.0

Remaining predictors after filtering: 62

```

['INTR_maximum_longevity_years', 'INTR_female_maturity_days',
'INTR_litter_or_clutch_size_n', 'INTR_litters_or_clutches_per_y',
'INTR_gestation_days', 'INTR_incubation_days', 'INTR_female_body_mass_g',
'INTR_birth_or_hatching_svl_cm', 'INTR_no_sex_maturity_days',
'INTR_gestation_incubation_days', 'INTR_litter_clutch_size',
'INTR_litters_clutches_per_year', 'INTR_interbirth_interval_years_1',
'INTR_birth_weight_g', 'INTR_metabolic_rate_w', 'total_locations',
'habitat_Artificial - Aquatic', 'habitat_Artificial - Terrestrial',
'habitat_Caves & Subterranean Habitats (non-aquatic)', 'habitat_Desert',
'habitat_Forest', 'habitat_Grassland', 'habitat_Introduced Vegetation',
'habitat_Marine Coastal/Supratidal', 'habitat_Marine Intertidal',
'habitat_Marine Neritic', 'habitat_Marine Oceanic', 'habitat_Other',
'habitat_Rocky Areas (e.g., inland cliffs, mountain peaks)', 'habitat_Savanna',
'habitat_Shrubland', 'habitat_Unknown', 'habitat_Wetlands (inland)',
'conservation_Education & awareness', 'conservation_Land/water management',
'conservation_Land/water protection', 'conservation_Law & policy',
'conservation_Livelihood, economic & other incentives', 'conservation_Species
management', 'threat1_Energy production & mining', 'threat1_Geological events',
'threat1_Invasive & other problematic species, genes & diseases']

```

```

[5]: #create final train and test datasets
X_train = pd.get_dummies(train[predictor_cols], drop_first=False)[cols_to_keep]
y_train = train['Extinct'].astype(int)

X_test = pd.get_dummies(test[predictor_cols], drop_first=False)[cols_to_keep]
y_test = test['Extinct'].astype(int)

scaler = StandardScaler()
X_train_scaled = pd.DataFrame(
    scaler.fit_transform(X_train),
    columns=X_train.columns,
    index=X_train.index
)
X_test_scaled = pd.DataFrame(
    scaler.transform(X_test),
    columns=X_test.columns,
    index=X_test.index
)

print("\nFinal training data shape:", X_train_scaled.shape)
print("Training response distribution:")
print(y_train.value_counts())

#fit Lasso logistic regression with cross-validation
print("Fitting Lasso logistic regression with cross-validation... (may take a_u
    ↳minute)")

#LogisticRegressionCV with L1 penalty (Lasso)
cv_model = LogisticRegressionCV(
    penalty='l1',
    solver='saga',
    cv=5,
    max_iter=10000,
    random_state=123,
    class_weight='balanced', #handle class imbalance
    scoring='roc_auc',
    n_jobs=-1
)

cv_model.fit(X_train_scaled, y_train)

print(f"Best regularization parameter (C): {cv_model.C_[0]:.6f}")
print(f"Equivalent lambda: {1/cv_model.C_[0]:.6f}")

#get non-zero coefficients
coef_df = pd.DataFrame({
    'Variable': X_train_scaled.columns,

```

```

        'Coefficient': cv_model.coef_[0],
        'Abs_Coefficient': np.abs(cv_model.coef_[0])
    })
coef_df = coef_df[coef_df['Coefficient'] != 0].sort_values('Abs_Coefficient',
    ↪ascending=False)

print("Non-zero coefficients (Variable Importance):")
print(coef_df)
print(f"\nNumber of selected variables: {len(coef_df)}")

#make predictions on training data
train_pred_proba = cv_model.predict_proba(X_train_scaled)[: , 1]

#make predictions on test data
test_pred_proba = cv_model.predict_proba(X_test_scaled)[: , 1]

#find optimal threshold from ROC curve
fpr, tpr, thresholds = roc_curve(y_test, test_pred_proba)
optimal_idx = np.argmax(tpr - fpr)
optimal_threshold = thresholds[optimal_idx]

print(f"Optimal threshold from ROC: {optimal_threshold:.4f}")

#make class predictions using optimal threshold
test_pred_class = (test_pred_proba > optimal_threshold).astype(int)

#calculate metrics
conf_mat = confusion_matrix(y_test, test_pred_class)
accuracy = accuracy_score(y_test, test_pred_class)
precision = precision_score(y_test, test_pred_class, zero_division=0)
recall = recall_score(y_test, test_pred_class)
f1 = f1_score(y_test, test_pred_class)
auc = roc_auc_score(y_test, test_pred_proba)

#display results
print("TEST SET RESULTS")
print("\nConfusion Matrix:")
print(conf_mat)
print("\nClassification Report:")
print(classification_report(y_test, test_pred_class, target_names=['Not_
    ↪Extinct', 'Extinct']))

print("\nPerformance Metrics:")
print(f"Accuracy: {accuracy:.4f}")
print(f"Precision: {precision:.4f}")
print(f"Recall: {recall:.4f}")
print(f"F1-score: {f1:.4f}")

```

```

print(f"AUC:          {auc:.4f}")

#plot ROC curve
plt.figure(figsize=(10, 6))
plt.plot(fpr, tpr, linewidth=2, label=f'ROC Curve (AUC = {auc:.4f})')
plt.plot([0, 1], [0, 1], 'k--', linewidth=1, label='Random Classifier')
plt.scatter(fpr[optimal_idx], tpr[optimal_idx], c='red', s=100,
            label=f'Optimal Threshold = {optimal_threshold:.3f}')
plt.xlabel('False Positive Rate (1 - Specificity)', fontsize=12)
plt.ylabel('True Positive Rate (Sensitivity)', fontsize=12)
plt.title('ROC Curve - Test Set', fontsize=14)
plt.legend(loc='lower right', fontsize=10)
plt.grid(alpha=0.3)
plt.tight_layout()
plt.show()

#model interpretation
print("MODEL INTERPRETATION:")
print(f"The model demonstrates excellent discriminatory power (AUC = {auc:.4f})_
    ↳with strong")
print(f"ability to identify extinct species (Recall = {recall:.4f}). It_
    ↳produces")
print(f"a few false positives (Precision = {precision:.4f}), which is expected_
    ↳given the severe")
print(f"class imbalance (only {y_test.mean()*100:.1f}% of species are extinct).
    ↳")

print("\nKey predictors of extinction risk: (OR = Odds Ratio)")

#interpret top predictors
for idx, row in coef_df.iterrows():
    odds_ratio = np.exp(row['Coefficient'])
    direction = "REDUCES" if row['Coefficient'] < 0 else "INCREASES"
    print(f"- {row['Variable']} {direction} extinction risk (OR = {odds_ratio:.
    ↳3f})")

#confusion matrix summary
tn, fp, fn, tp = conf_mat.ravel()
print("\nCONFUSION MATRIX SUMMARY")
print(f"True Negatives (correct non-extinct): {tn}")
print(f"False Positives (flagged for review): {fp}")
print(f"False Negatives (missed extinctions): {fn}")
print(f"True Positives (correct extinct): {tp}")
print(f"\nOut of {tp + fn} actual extinctions:")
print(f"    • Correctly identified: {tp} ({tp/(tp+fn)*100:.1f}%)")
print(f"    • Missed by model: {fn} ({fn/(tp+fn)*100:.1f}%)")
print(f"\nOut of {tn + fp} non-extinct species:")

```

```

print(f" • Correctly classified: {tn} ({tn/(tn+fp)*100:.1f}%")
print(f" • False alarms: {fp} ({fp/(tn+fp)*100:.1f}%")
print(f"\nThe model flags {fp + tp} species total for conservation priority,")
print(f"of which {tp} ({tp/(fp+tp)*100:.1f}%) are truly extinct/critically_
↪endangered.")

#create output dataframe
output_df = pd.DataFrame({
    'Species': test['Scientific'].values,
    'Predicted_Probability': test_pred_proba,
    'Predicted_Class': test_pred_class,
    'Actual_Class': y_test.values
})

print("Output predictions (first 6 rows):")
print(output_df.head(6))

#save to CSV
output_df.to_csv("comb_extinction_predictions_for_weibull_python.csv",
↪index=False)
print(f"\nFile saved with {len(output_df)} species")

```

Final training data shape: (3423, 62)

Training response distribution:

Extinct

0 3363

1 60

Name: count, dtype: int64

Fitting Lasso logistic regression with cross-validation... (may take a minute)

Best regularization parameter (C): 0.005995

Equivalent lambda: 166.810054

Non-zero coefficients (Variable Importance):

	Variable	Coefficient	Abs_Coefficient
48	conservation_Land/water management_0.0	0.387326	0.387326
49	conservation_Land/water management_1.0	-0.387326	0.387326
12	INTR_interbirth_interval_years_1	-0.361459	0.361459
51	conservation_Land/water protection_1.0	-0.331971	0.331971
50	conservation_Land/water protection_0.0	0.331971	0.331971
11	INTR_litters_clutches_per_year	0.211843	0.211843
25	habitat_Forest_1.0	-0.125778	0.125778
24	habitat_Forest_0.0	0.125778	0.125778
0	INTR_maximum_longevity_years	0.094134	0.094134

Number of selected variables: 9

Optimal threshold from ROC: 0.5431

TEST SET RESULTS

Confusion Matrix:

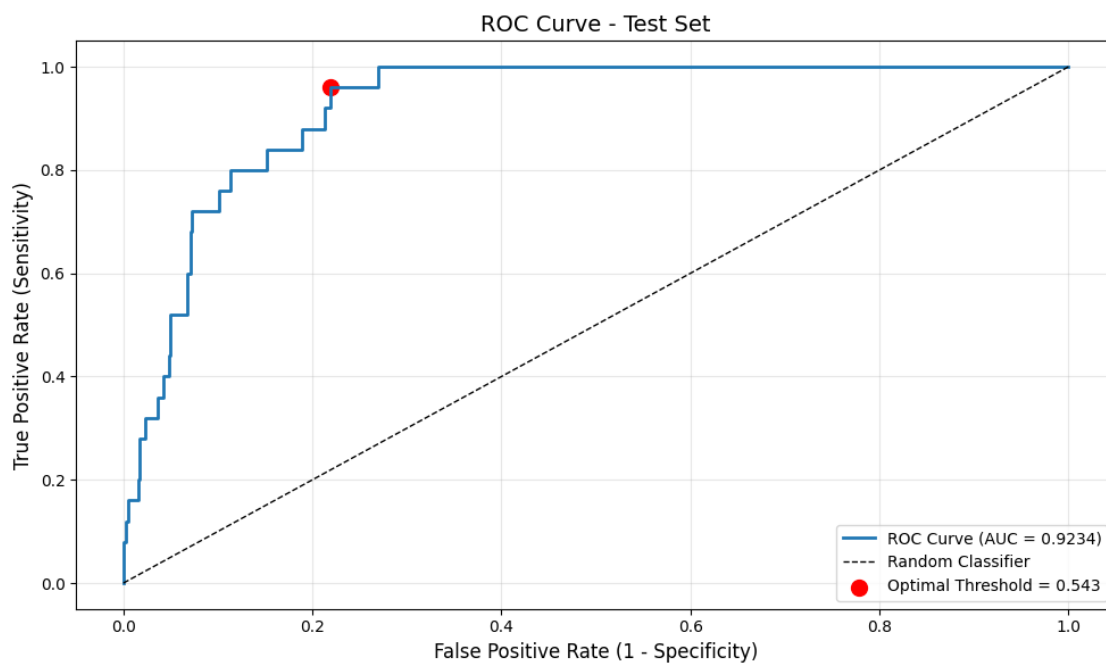
```
[[1127  315]
 [    2   23]]
```

Classification Report:

	precision	recall	f1-score	support
Not Extinct	1.00	0.78	0.88	1442
Extinct	0.07	0.92	0.13	25
accuracy			0.78	1467
macro avg	0.53	0.85	0.50	1467
weighted avg	0.98	0.78	0.86	1467

Performance Metrics:

Accuracy: 0.7839
Precision: 0.0680
Recall: 0.9200
F1-score: 0.1267
AUC: 0.9234



MODEL INTERPRETATION:

The model demonstrates excellent discriminatory power (AUC = 0.9234) with strong ability to identify extinct species (Recall = 0.9200). It produces

a few false positives (Precision = 0.0680), which is expected given the severe class imbalance (only 1.7% of species are extinct).

Key predictors of extinction risk: (OR = Odds Ratio)

- conservation_Land/water management_0.0 INCREASES extinction risk (OR = 1.473)
- conservation_Land/water management_1.0 REDUCES extinction risk (OR = 0.679)
- INTR_interbirth_interval_years_1 REDUCES extinction risk (OR = 0.697)
- conservation_Land/water protection_1.0 REDUCES extinction risk (OR = 0.718)
- conservation_Land/water protection_0.0 INCREASES extinction risk (OR = 1.394)
- INTR_litters_clutches_per_year INCREASES extinction risk (OR = 1.236)
- habitat_Forest_1.0 REDUCES extinction risk (OR = 0.882)
- habitat_Forest_0.0 INCREASES extinction risk (OR = 1.134)
- INTR_maximum_longevity_years INCREASES extinction risk (OR = 1.099)

CONFUSION MATRIX SUMMARY

True Negatives (correct non-extinct): 1127
 False Positives (flagged for review): 315
 False Negatives (missed extinctions): 2
 True Positives (correct extinct): 23

Out of 25 actual extinctions:

- Correctly identified: 23 (92.0%)
- Missed by model: 2 (8.0%)

Out of 1442 non-extinct species:

- Correctly classified: 1127 (78.2%)
- False alarms: 315 (21.8%)

The model flags 338 species total for conservation priority, of which 23 (6.8%) are truly extinct/critically endangered.

Output predictions (first 6 rows):

	Species	Predicted_Probability	Predicted_Class \
0	alsophylax tadjikiensis	0.466652	0
1	eulamprus leuraensis	0.043473	0
2	malacopteron magnum	0.547078	1
3	urocissa whiteheadi	0.534321	0
4	anolis marcanoi	0.306078	0
5	asio capensis	0.101049	0

	Actual_Class
0	0
1	0
2	0
3	0
4	0
5	0

File saved with 1467 species

```
[6]: for col in [
    "conservation_Land/water management",
    "conservation_Land/water protection",
    "INTR_interbirth_interval_years_1",
    "INTR_litters_clutches_per_year",
    "habitat_Forest",
    "INTR_maximum_longevity_years"
]:
    print(col, col in cols_to_keep)
```

```
conservation_Land/water management False
conservation_Land/water protection False
INTR_interbirth_interval_years_1 True
INTR_litters_clutches_per_year True
habitat_Forest False
INTR_maximum_longevity_years True
```

```
[7]: def predict_combined_logistic(lw_management, lw_protection, interbirth,
    ↪litters_clutches, forest, lifespan, species):

    #original row for species
    if species not in combined_data.index:
        return [float('nan')]

    #row w/ original predictor columns
    original_row = combined_data[predictor_cols].loc[[species]].copy()

    #one-hot encoding
    X_log_input = pd.get_dummies(original_row, drop_first=False)

    for col in cols_to_keep:
        if col not in X_log_input.columns:
            X_log_input[col] = 0

    X_log_input = X_log_input[cols_to_keep]

    #update categorical variables (they exist as dummy columns now)
    if 'conservation_Land/water management_0.0' in X_log_input.columns and \
        'conservation_Land/water management_1.0' in X_log_input.columns:
        X_log_input['conservation_Land/water management_0.0'] = 1 -
    ↪lw_management
        X_log_input['conservation_Land/water management_1.0'] = lw_management

    if 'conservation_Land/water protection_0.0' in X_log_input.columns and \
        'conservation_Land/water protection_1.0' in X_log_input.columns:
        X_log_input['conservation_Land/water protection_0.0'] = 1 -
    ↪lw_protection
```



```

X_log_input['conservation_Land/water protection_1.0'] = lw_protection

if 'habitat_Forest_0.0' in X_log_input.columns and \
    'habitat_Forest_1.0' in X_log_input.columns:
    X_log_input['habitat_Forest_0.0'] = 1 - forest
    X_log_input['habitat_Forest_1.0'] = forest

#update numeric predictors
if 'INTR_interbirth_interval_years_1' in X_log_input.columns:
    X_log_input['INTR_interbirth_interval_years_1'] = interbirth

if 'INTR_litters_clutches_per_year' in X_log_input.columns:
    X_log_input['INTR_litters_clutches_per_year'] = litters_clutches

if 'INTR_maximum_longevity_years' in X_log_input.columns:
    X_log_input['INTR_maximum_longevity_years'] = lifespan

#scale the data
X_log_input = pd.DataFrame(
    scaler.transform(X_log_input),
    columns=X_log_input.columns,
    index=X_log_input.index
)

#predict
predictions = cv_model.predict_proba(X_log_input)[: , 1]
predicted_class = (predictions > optimal_threshold).astype(int)

if predicted_class[0] == 1:
    class_label_tab = "Extinct"
else:
    class_label_tab = "Not Extinct"

return class_label_tab

print(predict_combined_logistic(0, 0, 1, 0.1, 0, 1, 'asio capensis'))

```

[nan]

1 RISK SCORES ADDED

```

[8]: comb_data_w_risk = pd.merge(combined_data, risk, left_on='Scientific',
    ↪right_on='Species', how='inner').drop('Species', axis=1).drop('Extinct_y',
    ↪axis=1).rename(columns={'Extinct_x': 'Extinct'})

```

```

[9]: #split into train and test sets with stratification
    np.random.seed(123)

```

```

train_risk, test_risk = train_test_split(comb_data_w_risk, test_size=0.3,
                                         stratify=comb_data_w_risk['Extinct'],
                                         random_state=123)

#double check proportions
print("Proportion of extinct in full data:", comb_data_w_risk['Extinct'].
      ↪value_counts(normalize=True))
print("\nProportion of extinct in train:", train_risk['Extinct'].
      ↪value_counts(normalize=True))
print("\nProportion of extinct in test:", test_risk['Extinct'].
      ↪value_counts(normalize=True))

#non-predictor columns
exclude_cols_risk = ['Extinct', 'Scientific', 'population_trend.description',
                    ↪'Survival_Probability', 'Risk Index']

#predictor columns
predictor_cols_risk = [col for col in train_risk.columns if col not in
                    ↪exclude_cols_risk]

#prep data for variance filtering
X_train_temp_risk = pd.get_dummies(train_risk[predictor_cols_risk],
    ↪drop_first=False)
X_train_temp_risk = X_train_temp_risk.apply(pd.to_numeric, errors='coerce').
    ↪fillna(0)

#near-zero variance filtering
selector_risk = VarianceThreshold(threshold=0.001) #low threshold for
    ↪near-zero variance
selector_risk.fit(X_train_temp_risk)

#get columns to keep
variance_mask_risk = selector_risk.get_support()
cols_to_keep_risk = X_train_temp_risk.columns[variance_mask_risk].tolist()

#ID removed columns
removed_cols_risk = X_train_temp_risk.columns[~variance_mask_risk].tolist()
print("Zero/near-zero variance predictors removed:")
for col in removed_cols_risk:
    print(f" - {col}")

print(f"\nRemaining predictors after filtering: {len(cols_to_keep_risk)}")

```

```

Proportion of extinct in full data: Extinct
0    0.982618
1    0.017382
Name: proportion, dtype: float64

```

Proportion of extinct in train: Extinct

0 0.982472

1 0.017528

Name: proportion, dtype: float64

Proportion of extinct in test: Extinct

0 0.982958

1 0.017042

Name: proportion, dtype: float64

Zero/near-zero variance predictors removed:

- habitat_Introduced Vegetation_0.0
- habitat_Introduced Vegetation_1.0
- habitat_Unknown_0.0
- habitat_Unknown_1.0
- threat1_Geological events_0.0
- threat1_Geological events_1.0

Remaining predictors after filtering: 63

```
[10]: #create final train and test datasets
X_train_risk = pd.get_dummies(train_risk[predictor_cols_risk],
                               drop_first=False)[cols_to_keep_risk]
y_train_risk = train_risk['Extinct'].astype(int)

X_test_risk = pd.get_dummies(test_risk[predictor_cols_risk],
                              drop_first=False)[cols_to_keep_risk]
y_test_risk = test_risk['Extinct'].astype(int)

scaler = StandardScaler()
X_train_scaled_risk = pd.DataFrame(
    scaler.fit_transform(X_train_risk),
    columns=X_train_risk.columns,
    index=X_train_risk.index
)
X_test_scaled_risk = pd.DataFrame(
    scaler.transform(X_test_risk),
    columns=X_test_risk.columns,
    index=X_test_risk.index
)

print("\nFinal training data shape:", X_train_scaled_risk.shape)
print("Training response distribution:")
print(y_train_risk.value_counts())

#fit Lasso logistic regression with cross-validation
```

```

print("Fitting Lasso logistic regression with cross-validation... (may take a
↳minute)")

#LogisticRegressionCV with L1 penalty (Lasso)
cv_model_risk = LogisticRegressionCV(
    penalty='l1',
    solver='saga',
    cv=5,
    max_iter=10000,
    random_state=123,
    class_weight='balanced', #handle class imbalance
    scoring='roc_auc',
    n_jobs=-1
)

cv_model_risk.fit(X_train_scaled_risk, y_train_risk)

print(f"Best regularization parameter (C): {cv_model_risk.C_[0]:.6f}")
print(f"Equivalent lambda: {1/cv_model_risk.C_[0]:.6f}")

#get non-zero coefficients
coef_df_risk = pd.DataFrame({
    'Variable': X_train_risk.columns,
    'Coefficient': cv_model_risk.coef_[0],
    'Abs_Coefficient': np.abs(cv_model_risk.coef_[0])
})
coef_df_risk = coef_df_risk[coef_df_risk['Coefficient'] != 0].
↳sort_values('Abs_Coefficient', ascending=False)

print("Non-zero coefficients (Variable Importance):")
print(coef_df_risk)
print(f"\nNumber of selected variables: {len(coef_df_risk)}")

#make predictions on training data
train_pred_proba_risk = cv_model_risk.predict_proba(X_train_scaled_risk)[: , 1]

#make predictions on test data
test_pred_proba_risk = cv_model_risk.predict_proba(X_test_scaled_risk)[: , 1]

#find optimal threshold from ROC curve
fpr_risk, tpr_risk, thresholds_risk = roc_curve(y_test_risk,
↳test_pred_proba_risk)
optimal_idx_risk = np.argmax(tpr_risk - fpr_risk)
optimal_threshold_risk = thresholds_risk[optimal_idx_risk]

print(f"Optimal threshold from ROC: {optimal_threshold_risk:.4f}")

```

```

#make class predictions using optimal threshold
test_pred_class_risk = (test_pred_proba_risk > optimal_threshold_risk).
    ↪astype(int)

#calculate metrics
conf_mat_risk = confusion_matrix(y_test_risk, test_pred_class_risk)
accuracy_risk = accuracy_score(y_test_risk, test_pred_class_risk)
precision_risk = precision_score(y_test_risk, test_pred_class_risk,
    ↪zero_division=0)
recall_risk = recall_score(y_test_risk, test_pred_class_risk)
f1_risk = f1_score(y_test_risk, test_pred_class_risk)
auc_risk = roc_auc_score(y_test_risk, test_pred_proba_risk)

#display results
print("TEST SET RESULTS")
print("\nConfusion Matrix:")
print(conf_mat_risk)
print("\nClassification Report:")
print(classification_report(y_test_risk, test_pred_class_risk,
    ↪target_names=['Not Extinct', 'Extinct']))

print("\nPerformance Metrics:")
print(f"Accuracy:  {accuracy_risk:.4f}")
print(f"Precision: {precision_risk:.4f}")
print(f"Recall:    {recall_risk:.4f}")
print(f"F1-score:  {f1_risk:.4f}")
print(f"AUC:      {auc_risk:.4f}")

#plot ROC curve
plt.figure(figsize=(10, 6))
plt.plot(fpr_risk, tpr_risk, linewidth=2, label=f'ROC Curve (AUC = {auc_risk:.
    ↪4f})')
plt.plot([0, 1], [0, 1], 'k--', linewidth=1, label='Random Classifier')
plt.scatter(fpr_risk[optimal_idx_risk], tpr_risk[optimal_idx_risk], c='red',
    ↪s=100,
            label=f'Optimal Threshold = {optimal_threshold_risk:.3f}')
plt.xlabel('False Positive Rate (1 - Specificity)', fontsize=12)
plt.ylabel('True Positive Rate (Sensitivity)', fontsize=12)
plt.title('ROC Curve - Test Set', fontsize=14)
plt.legend(loc='lower right', fontsize=10)
plt.grid(alpha=0.3)
plt.tight_layout()
plt.show()

#model interpretation
print("MODEL INTERPRETATION:")

```

```

print(f"The model demonstrates excellent discriminatory power (AUC = {auc_risk:.4f}) with strong")
print(f"ability to identify extinct species (Recall = {recall_risk:.4f}). It produces")
print(f"a few false positives (Precision = {precision_risk:.4f}), which is expected given the severe")
print(f"class imbalance (only {y_test_risk.mean()*100:.1f}% of species are extinct).")

print("\nKey predictors of extinction risk: (OR = Odds Ratio)")

#interpret top predictors
for idx_risk, row_risk in coef_df_risk.iterrows():
    odds_ratio = np.exp(row_risk['Coefficient'])
    direction = "REDUCES" if row_risk['Coefficient'] < 0 else "INCREASES"
    print(f"- {row_risk['Variable']} {direction} extinction risk (OR = {odds_ratio:.3f})")

#confusion matrix summary
tn_risk, fp_risk, fn_risk, tp_risk = conf_mat_risk.ravel()
print("\nCONFUSION MATRIX SUMMARY")
print(f"True Negatives (correct non-extinct): {tn_risk}")
print(f"False Positives (flagged for review): {fp_risk}")
print(f"False Negatives (missed extinctions): {fn_risk}")
print(f"True Positives (correct extinct): {tp_risk}")
print(f"\nOut of {tp_risk + fn_risk} actual extinctions:")
print(f"    • Correctly identified: {tp_risk} ({tp_risk/(tp_risk+fn_risk)*100:.1f}%)")
print(f"    • Missed by model: {fn_risk} ({fn_risk/(tp_risk+fn_risk)*100:.1f}%)")
print(f"\nOut of {tn_risk + fp_risk} non-extinct species:")
print(f"    • Correctly classified: {tn_risk} ({tn_risk/(tn_risk+fp_risk)*100:.1f}%)")
print(f"    • False alarms: {fp_risk} ({fp_risk/(tn_risk+fp_risk)*100:.1f}%)")
print(f"\nThe model flags {fp_risk + tp_risk} species total for conservation priority,")
print(f"of which {tp_risk} ({tp_risk/(fp_risk+tp_risk)*100:.1f}%) are truly extinct/critically endangered.")

#create output dataframe
output_df_risk = pd.DataFrame({
    'Species': test_risk['Scientific'].values,
    'Predicted_Probability': test_pred_proba_risk,
    'Predicted_Class': test_pred_class_risk,
    'Actual_Class': y_test_risk.values
})

```

```

print("Output predictions (first 6 rows):")
print(output_df_risk.head(6))

#save to CSV
output_df_risk.to_csv("comb_predictions_for_weibull_python_w_risk.csv",
    ↪index=False)
print(f"\nFile saved with {len(output_df_risk)} species")

```

Final training data shape: (3423, 63)

Training response distribution:

Extinct

0 3363

1 60

Name: count, dtype: int64

Fitting Lasso logistic regression with cross-validation... (may take a minute)

Best regularization parameter (C): 0.046416

Equivalent lambda: 21.544347

Non-zero coefficients (Variable Importance):

	Variable	Coefficient \
16	Risk Score	1.690843
9	INTR_gestation_incubation_days	-1.258233
49	conservation_Land/water management_0.0	0.562819
50	conservation_Land/water management_1.0	-0.562819
11	INTR_litters_clutches_per_year	-0.487432
52	conservation_Land/water protection_1.0	-0.298103
51	conservation_Land/water protection_0.0	0.298103
8	INTR_no_sex_maturity_days	0.259370
57	conservation_Species management_0.0	0.210120
58	conservation_Species management_1.0	-0.210120
0	INTR_maximum_longevity_years	0.205699
27	habitat_Grassland_0.0	0.196734
28	habitat_Grassland_1.0	-0.196734
33	habitat_Marine Neritic_0.0	0.186296
34	habitat_Marine Neritic_1.0	-0.186296
31	habitat_Marine Intertidal_0.0	-0.117755
32	habitat_Marine Intertidal_1.0	0.117755
46	habitat_Wetlands (inland)_1.0	-0.093451
45	habitat_Wetlands (inland)_0.0	0.093451
3	INTR_litters_or_clutches_per_y	0.093178
26	habitat_Forest_1.0	-0.083889
25	habitat_Forest_0.0	0.083889
62	threat1_Invasive & other problematic species, ...	-0.046127
61	threat1_Invasive & other problematic species, ...	0.046127
20	habitat_Artificial - Terrestrial_1.0	-0.036285
19	habitat_Artificial - Terrestrial_0.0	0.036285
30	habitat_Marine Coastal/Supratidal_1.0	0.032406
29	habitat_Marine Coastal/Supratidal_0.0	-0.032406

35	habitat_Marine Oceanic_0.0	0.002067
36	habitat_Marine Oceanic_1.0	-0.002067
48	conservation_Education & awareness_1.0	-0.000406
47	conservation_Education & awareness_0.0	0.000406

	Abs_Coefficient
16	1.690843
9	1.258233
49	0.562819
50	0.562819
11	0.487432
52	0.298103
51	0.298103
8	0.259370
57	0.210120
58	0.210120
0	0.205699
27	0.196734
28	0.196734
33	0.186296
34	0.186296
31	0.117755
32	0.117755
46	0.093451
45	0.093451
3	0.093178
26	0.083889
25	0.083889
62	0.046127
61	0.046127
20	0.036285
19	0.036285
30	0.032406
29	0.032406
35	0.002067
36	0.002067
48	0.000406
47	0.000406

Number of selected variables: 32
 Optimal threshold from ROC: 0.3700
 TEST SET RESULTS

Confusion Matrix:
 [[1210 232]
 [1 24]]

Classification Report:

	precision	recall	f1-score	support
Not Extinct	1.00	0.84	0.91	1442
Extinct	0.09	0.96	0.17	25
accuracy			0.84	1467
macro avg	0.55	0.90	0.54	1467
weighted avg	0.98	0.84	0.90	1467

Performance Metrics:

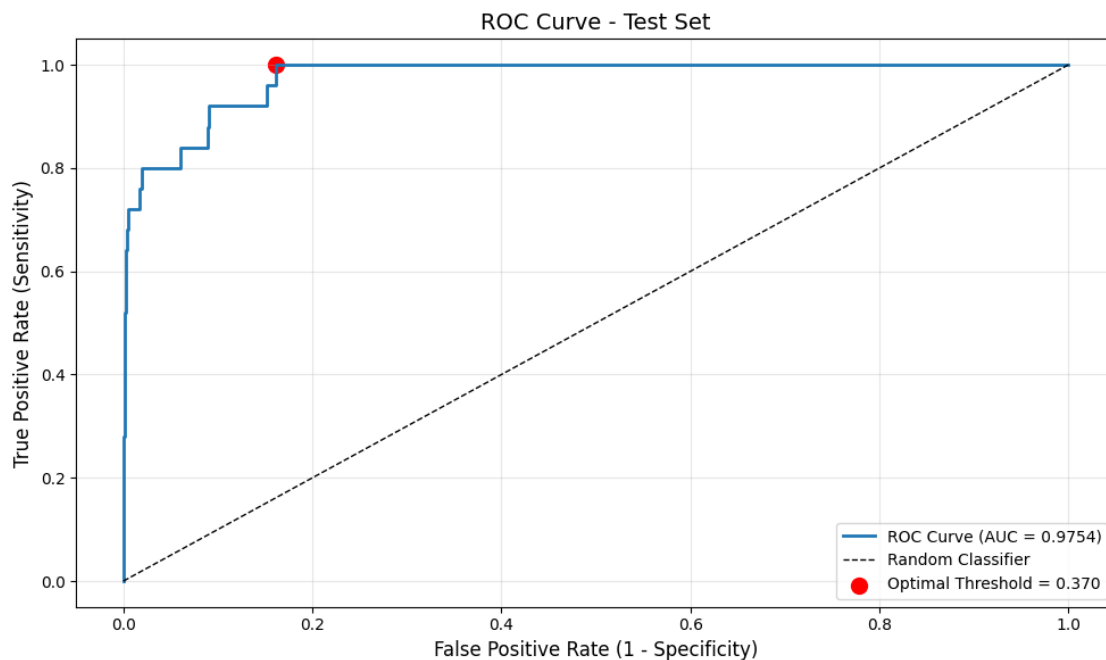
Accuracy: 0.8412

Precision: 0.0938

Recall: 0.9600

F1-score: 0.1708

AUC: 0.9754



MODEL INTERPRETATION:

The model demonstrates excellent discriminatory power (AUC = 0.9754) with strong ability to identify extinct species (Recall = 0.9600). It produces a few false positives (Precision = 0.0938), which is expected given the severe class imbalance (only 1.7% of species are extinct).

Key predictors of extinction risk: (OR = Odds Ratio)

- Risk Score INCREASES extinction risk (OR = 5.424)
- INTR_gestation_incubation_days REDUCES extinction risk (OR = 0.284)

- conservation_Land/water management_0.0 INCREASES extinction risk (OR = 1.756)
- conservation_Land/water management_1.0 REDUCES extinction risk (OR = 0.570)
- INTR_litters_clutches_per_year REDUCES extinction risk (OR = 0.614)
- conservation_Land/water protection_1.0 REDUCES extinction risk (OR = 0.742)
- conservation_Land/water protection_0.0 INCREASES extinction risk (OR = 1.347)
- INTR_no_sex_maturity_days INCREASES extinction risk (OR = 1.296)
- conservation_Species management_0.0 INCREASES extinction risk (OR = 1.234)
- conservation_Species management_1.0 REDUCES extinction risk (OR = 0.810)
- INTR_maximum_longevity_years INCREASES extinction risk (OR = 1.228)
- habitat_Grassland_0.0 INCREASES extinction risk (OR = 1.217)
- habitat_Grassland_1.0 REDUCES extinction risk (OR = 0.821)
- habitat_Marine Neritic_0.0 INCREASES extinction risk (OR = 1.205)
- habitat_Marine Neritic_1.0 REDUCES extinction risk (OR = 0.830)
- habitat_Marine Intertidal_0.0 REDUCES extinction risk (OR = 0.889)
- habitat_Marine Intertidal_1.0 INCREASES extinction risk (OR = 1.125)
- habitat_Wetlands (inland)_1.0 REDUCES extinction risk (OR = 0.911)
- habitat_Wetlands (inland)_0.0 INCREASES extinction risk (OR = 1.098)
- INTR_litters_or_clutches_per_y INCREASES extinction risk (OR = 1.098)
- habitat_Forest_1.0 REDUCES extinction risk (OR = 0.920)
- habitat_Forest_0.0 INCREASES extinction risk (OR = 1.088)
- threat1_Invasive & other problematic species, genes & diseases_1.0 REDUCES extinction risk (OR = 0.955)
- threat1_Invasive & other problematic species, genes & diseases_0.0 INCREASES extinction risk (OR = 1.047)
- habitat_Artificial - Terrestrial_1.0 REDUCES extinction risk (OR = 0.964)
- habitat_Artificial - Terrestrial_0.0 INCREASES extinction risk (OR = 1.037)
- habitat_Marine Coastal/Supratidal_1.0 INCREASES extinction risk (OR = 1.033)
- habitat_Marine Coastal/Supratidal_0.0 REDUCES extinction risk (OR = 0.968)
- habitat_Marine Oceanic_0.0 INCREASES extinction risk (OR = 1.002)
- habitat_Marine Oceanic_1.0 REDUCES extinction risk (OR = 0.998)
- conservation_Education & awareness_1.0 REDUCES extinction risk (OR = 1.000)
- conservation_Education & awareness_0.0 INCREASES extinction risk (OR = 1.000)

CONFUSION MATRIX SUMMARY

True Negatives (correct non-extinct): 1210
 False Positives (flagged for review): 232
 False Negatives (missed extinctions): 1
 True Positives (correct extinct): 24

Out of 25 actual extinctions:

- Correctly identified: 24 (96.0%)
- Missed by model: 1 (4.0%)

Out of 1442 non-extinct species:

- Correctly classified: 1210 (83.9%)
- False alarms: 232 (16.1%)

The model flags 256 species total for conservation priority,

of which 24 (9.4%) are truly extinct/critically endangered.

Output predictions (first 6 rows):

	Species	Predicted_Probability	Predicted_Class	\
0	alsophylax tadjikiensis	0.175985	0	
1	eulamprus leuraensis	0.000948	0	
2	malacopteron magnum	0.456511	1	
3	urocissa whiteheadi	0.586281	1	
4	anolis marcanoi	0.157492	0	
5	asio capensis	0.010266	0	

	Actual_Class
0	0
1	0
2	0
3	0
4	0
5	0

File saved with 1467 species

[]:

[]: